Foodies — REST API Endpoints

Web Programming 1 · Stage 3 · Julien Matar, Peter Nehmeh, Freddy Rayess

> **Base URL:** `http://localhost:4040/api` · All requests and responses are JSON. The website in `view/` is served at `http://localhost:4040`.
>
> **Status codes:** 200 OK, 201 Created, 204 Deleted, 400 invalid input, 401 not logged in, 403 not allowed, 404 not found, 409 conflict (already exists), 500 server error. Errors are always `{ "error": "message" }`.
>
> **Access:** Public = no login · User = logged in (JWT in the httpOnly cookie `token`) · Owner = the user who wrote the review · Admin = admin role.
>
> **Sort values** for `/restaurants?sort=`: `authentic`, `mix` (fusion first), `modern`, `price_asc`, `price_desc`, `top_rated`. **Styles:** `heritage`, `authentic`, `fusion`, `modern`. **Allergens:** `gluten`, `dairy`, `nuts`, `sesame`, `fish`, `eggs`. **Meat:** none, `lamb`, `beef`, `chicken`, `fish`.

Method	Endpoint	Access	Purpose / body
Auth (6)			
POST	`/api/auth/register`	Public	Create an account and log in { name, email, password }
POST	`/api/auth/login`	Public	Log in, sets the JWT cookie { email, password }
POST	`/api/auth/logout`	User	Clear the cookie answers 204
GET	`/api/auth/me`	User	The logged-in user
GET	`/api/auth/google`	Public	Start Google sign-in needs GOOGLE_* in .env
GET	`/api/auth/google/callback`	Public	Finish Google sign-in
Dishes (5)			
GET	`/api/dishes?search=&category=&vegetarian=&vegan=&gluten_free=&without=`	Public	List / search dishes category = id · vegetarian = true|false · vegan, gluten_free = true · without = allergens, e.g. nuts,sesame
GET	`/api/dishes/:id`	Public	One dish (ingredients, diet, allergens) + the restaurants serving it and their prices
POST	`/api/dishes`	Admin	Create a dish { name_en, name_ar, description, category_id, is_vegetarian, weather } + optional { slug, ingredients[], is_vegan, is_gluten_free, allergens[], meat, served_raw, good_to_know, first_timer }
PUT	`/api/dishes/:id`	Admin	Replace a dish same body
DELETE	`/api/dishes/:id`	Admin	Delete a dish
Restaurants (5)			
GET	`/api/restaurants?dish=&region=&style=&sort=`	Public	Filter and sort restaurants, with average rating, tradition score and review count dish, region = ids · with dish= only the places where it is available, with its price_usd
GET	`/api/restaurants/:id`	Public	One restaurant + scores + menu with prices
POST	`/api/restaurants`	Admin	Create { name, region_id, address, phone, lat, lng, price_level 1-4, style }
PUT	`/api/restaurants/:id`	Admin	Replace a restaurant same body
DELETE	`/api/restaurants/:id`	Admin	Delete a restaurant
Menu: which restaurant serves which dish (3)			
POST	`/api/restaurants/:id/dishes`	Admin	Put a dish on the menu { dish_id, price_usd, available }
PUT	`/api/restaurants/:id/dishes/:dishId`	Admin	Change price or availability { price_usd?, available? }
DELETE	`/api/restaurants/:id/dishes/:dishId`	Admin	Take a dish off the menu
Reviews (4)			
GET	`/api/restaurants/:id/reviews`	Public	Reviews of a restaurant, newest first
POST	`/api/restaurants/:id/reviews`	User	Add a review (one per restaurant) { rating 1-5, tradition_rating 1-5, comment }
PUT	`/api/reviews/:id`	Owner	Edit own review same body
DELETE	`/api/reviews/:id`	Owner or admin	Delete a review
Favorites (3)			
GET	`/api/favorites`	User	My favorite dishes, the last one first
POST	`/api/favorites/:dishId`	User	Add a favorite
DELETE	`/api/favorites/:dishId`	User	Remove a favorite
Lookups (2)			
GET	`/api/regions`	Public	List regions
GET	`/api/categories`	Public	List dish categories
Utilities (2)			
GET	`/api/dish-of-the-day`	Public	A dish that suits today's weather in Beirut Open-Meteo
GET	`/api/rates`	Public	1 USD in LBP and EUR cached 1 hour · estimated: true when the rates service is down
Chatbot (planned, not built yet) (2)			
POST	`/api/chat`	Public, rate-limited	Ask a question, get an answer + matching restaurants { question }
GET	`/api/admin/chat-logs`	Admin	Questions asked to the chatbot

Total: 32 endpoints · 30 built · the 2 chatbot endpoints are planned for later.